About The AdaGamma Optimizer

An Adam extension using higher-order gradient moments

Apurv Bhushan
Optimization Course - FM216, Monsoon 2025

What this document covers

This log explains the problem, the math behind AdaGamma, how the code implements it, how the tests were set up, what the results show, and what the limitations are. The aim is to explain the project clearly rather than to document every line of code.

Contents

1 Project overview **4**

1.1 The basic idea 4

2 Why did we work on a new optimizer? **4**

2.1 What do skewness and kurtosis mean? 5

3 The mathematics behind AdaGamma **5**

3.1 Notation 5

3.2 Step 1: Calculate the gradient 5

3.3 Step 2: First and second moments 5

3.4 Step 3: Centralise the gradient 6

3.5 Step 4: Bias correction 6

3.6 Step 5: Estimate skewness and excess kurtosis 6

3.7 Step 6: Calculate the modulation factor 7

3.8 Step 7: Smooth the modulation factor 7

3.9 Step 8: Update the parameters 7

4 How the math became code **7**

4.1 Initialising the optimizer 8

4.2 Updating the moments 8

4.3 Calculating gamma and updating parameters 8

4.4 A difference between the poster and the final implementation 9

5 Hyperparameters and tuning **9**

5.1 Why tuning was needed 10

5.2 How the synthetic-problem search works 10

6 Testing the optimizer **11**

6.1 Well-conditioned quadratic 11

6.2 Ill-conditioned quadratic 12

6.3 Rosenbrock function 13

6.4 MNIST digit classification 13

7 What the results tell us **14**

8 Limitations and possible improvements **14**

8.1 Performance on neural-network training 15

8.2 Extra computation and memory 15
8.3 More hyperparameters 15

9 My contribution and what I learned **15**

10 Notes about the submitted files **15**

11 Conclusion **16**

1 Project overview

The task was to build an optimizer and compare it against common optimizers on different problems as part of my ***Optimization*** course. The tests included quadratic functions, the Rosenbrock function and MNIST digit classification.

My main contribution was the mathematical development of the optimizer. I worked on the update rule and the role of the extra gradient statistics. I then helped turn that idea into code and worked on testing and tuning the hyperparameters. The final report also records that all four members took part in the idea and testing, while the report and poster had separate main contributors.

1.1 The basic idea

Most adaptive optimizers use information about the average gradient and the average squared gradient. AdaGamma keeps this Adam-style base, but adds two more statistics:

- the third central moment, which is used to estimate **skewness**;
- the fourth central moment, which is used to estimate **kurtosis**.

These statistics describe the shape of the gradient distribution. AdaGamma uses them to calculate a factor called γ. This factor scales the normal Adam-style update.

In simple terms:

1. Calculate the usual Adam quantities.
2. Track how asymmetric and heavy-tailed the recent gradients are.
3. Use those measurements to calculate a scaling factor.
4. Smooth that factor so it does not change too suddenly.
5. Apply the scaled update to the model parameters.

2 Why did we work on a new optimizer?

Adam is widely used because it adjusts the step size using information from past gradients. It tracks:

- a moving average of gradients;
- a moving average of squared gradients.

These are related to the first and second moments.

However, two gradient sequences can have similar averages and variances but very different shapes. One may be fairly symmetric, while another may contain strong asymmetry or occasional large values. Adam's first and second moments do not directly describe these differences.

Our project idea was to add information about the shape of the gradients. We used skewness and kurtosis as two ways to describe that shape.

2.1 What do skewness and kurtosis mean?

Skewness describes asymmetry. A positive or negative value indicates that the distribution has a longer tail in one direction.

Kurtosis describes the weight of the tails relative to a normal distribution. In our implementation, we subtract 3 from kurtosis, giving **excess kurtosis**. A value near zero corresponds to the normal-distribution reference value; a positive value indicates heavier tails.

These values are not direct measurements of the loss function's curvature. They describe the recent gradient values. The report sometimes describes the method as curvature-aware, but the actual quantities used in the code are gradient-distribution statistics.

3 The mathematics behind AdaGamma

This section follows the final mathematical description in the project report and the implementation in `adagamma.py`. The presentation contains a shorter version of the algorithm. One important difference between the short presentation and the final code is explained in Section 4.

3.1 Notation

The main symbols are:

- θ_t: model parameters at iteration t;
- g_t: gradient at iteration t;
- α: learning rate;
- $\beta_1, \beta_2, \beta_3, \beta_4$: decay rates for the four moving averages;
- a: weight applied to skewness;
- b: weight applied to excess kurtosis;
- η: smoothing factor for γ;
- ϵ: a small number used for numerical stability.

3.2 Step 1: Calculate the gradient

At the current parameter values, calculate the gradient of the loss:

$$g_t = \nabla_\theta f(\theta_{t-1}).$$

The gradient tells us the direction in which the loss increases fastest. The optimizer moves in the opposite direction to reduce the loss.

3.3 Step 2: First and second moments

AdaGamma first calculates the same two moving averages used by Adam:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Here, m_t tracks the average gradient and v_t tracks the average squared gradient. The square is element-wise when the parameter has multiple elements.

The first quantity gives the direction of the update. The second helps scale the update based on the recent size of the gradients.

3.4 Step 3: Centralise the gradient

For the higher-order moments, the final implementation first calculates:

$$c_t = g_t - m_{t-1}.$$

This is the current gradient minus the previous moving-average estimate. It is used as an approximation of a deviation from the recent mean.

The third and fourth moments are then updated as:

$$s_t = \beta_3 s_{t-1} + (1-\beta_3)c_t^3,$$

$$k_t = \beta_4 k_{t-1} + (1-\beta_4)c_t^4.$$

The third power preserves the sign and therefore carries information about asymmetry. The fourth power is non-negative and gives more weight to large deviations.

3.5 Step 4: Bias correction

All four moving averages start at zero. Early in training, this makes them smaller than the values they are trying to estimate. We correct for this initial bias:

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1-\beta_2^t},$$

$$\hat{s}_t = \frac{s_t}{1-\beta_3^t}, \qquad \hat{k}_t = \frac{k_t}{1-\beta_4^t}.$$

This is the same general bias-correction idea used in Adam.

3.6 Step 5: Estimate skewness and excess kurtosis

First calculate a scale estimate:

$$\sigma_t = \sqrt{\hat{v}_t} + \epsilon.$$

Then calculate:

$$\text{skew}_t = \frac{\hat{s}_t}{\sigma_t^3},$$

$$\text{kurt}_t = \frac{\hat{k}_t}{\sigma_t^4}, \qquad \text{excess}_t = \text{kurt}_t - 3.$$

The division by the standard-deviation estimate makes these statistics dimensionless. The subtraction of 3 gives excess kurtosis.

3.7 Step 6: Calculate the modulation factor

The raw modulation value is:

$$R_t = 1 + a \text{ skew}_t + b \text{ excess}_t .$$

We limit it to a chosen range:

$$R_t^{\text{clip}} = \text{clip}(R_t, \gamma_{\min}, \gamma_{\max}).$$

Clipping prevents this value from becoming too small or too large. The instantaneous modulation factor is its inverse:

$$\gamma_t^{\text{inst}} = \frac{1}{R_t^{\text{clip}}}.$$

The coefficients a and b control how strongly skewness and excess kurtosis affect the update.

3.8 Step 7: Smooth the modulation factor

The instantaneous value can change sharply from one iteration to the next. To reduce sudden changes, the code uses an exponential moving average:

$$\gamma_t = \eta\gamma_{t-1} + (1 - \eta)\gamma_t^{\text{inst}}.$$

A larger η gives more weight to the previous value, so γ changes more slowly.

3.9 Step 8: Update the parameters

The final update is:

$$\theta_t = \theta_{t-1} - \alpha\gamma_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

This is the Adam-style update multiplied by γ_t.

If γ_t is smaller, the update becomes smaller. If it is larger, the update becomes larger. The exact value depends on the recent skewness and excess kurtosis estimates, the coefficients, and the clipping limits.

4 How the math became code

The main implementation is in `adagamma.py`. It is written as a PyTorch optimizer class. This means it can be passed to a PyTorch training loop in a similar way to Adam.

The main parts of the code are:

1. `__init__`: stores the learning rate and hyperparameters.
2. `step`: reads the gradients, updates the moving averages, calculates γ, and updates the parameters.
3. `state`: stores each parameter's running values between training steps.

4.1 Initialising the optimizer

The code sets default values for the parameters. The defaults in the Python implementation are not exactly the same as the values listed in every report section, because the values were adjusted during testing.

Listing 1: The AdaGamma constructor (excerpt)

```
class AdaGamma(Optimizer):
    def __init__(self, params,
                 lr=1e-3,
                 beta1=0.9, beta2=0.999,
                 beta3=0.95, beta4=0.99,
                 a=0.2, b=0.05,
                 gamma_min=0.1, gamma_max=2.0,
                 gamma_smooth=0.9,
                 eps=1e-8):
        defaults = dict(
            lr=lr,
            beta1=beta1, beta2=beta2,
            beta3=beta3, beta4=beta4,
            a=a, b=b,
            gamma_min=gamma_min, gamma_max=gamma_max,
            gamma_smooth=gamma_smooth,
            eps=eps
        )
        super(AdaGamma, self).__init__(params, defaults)
```

4.2 Updating the moments

The next excerpt shows the core of the optimizer. The variable `centered` is the current gradient minus the previous first-moment estimate. This is what the code uses for the third and fourth moments.

Listing 2: Moment updates and bias correction (from adagamma.py)

```
m_prev = state["m"]

state["m"] = b1 * state["m"] + (1 - b1) * g
state["v"] = b2 * state["v"] + (1 - b2) * (g * g)

centered = g - m_prev
state["s"] = b3 * state["s"] + (1 - b3) * centered**3
state["k"] = b4 * state["k"] + (1 - b4) * centered**4

m_hat = state["m"] / (1 - b1**t)
v_hat = state["v"] / (1 - b2**t)
s_hat = state["s"] / (1 - b3**t)
k_hat = state["k"] / (1 - b4**t)
```

4.3 Calculating gamma and updating parameters

Listing 3: Modulation and parameter update (from adagamma.py)

```
std_hat = torch.sqrt(v_hat) + eps
skew = s_hat / (std_hat**3)
```

```
kurt = k_hat / (std_hat**4)
excess = kurt - 3.0

raw = 1.0 + a * skew + b * excess
raw = torch.clamp(raw, gmin, gmax)
gamma_new = 1.0 / raw

state["gamma"] = (
    g_smooth * state["gamma"]
    + (1 - g_smooth) * gamma_new
)

step = lr * state["gamma"] * m_hat / std_hat
p.add_(-step)
```

4.4 A difference between the poster and the final implementation

The poster's compact pseudocode writes the third and fourth moment updates using g_t^3 and g_t^4. The final report and the Python implementation use a centralised quantity, $c_t = g_t - m_{t-1}$, instead. The code also explicitly divides by powers of the standard-deviation estimate to calculate skewness and kurtosis. This log describes the implementation that is actually present in `adagamma.py`.

There is a second small implementation detail: the code keeps a separate γ value for each parameter element (because the state tensors have the same shape as the parameter). Therefore, in the code, the modulation is element-wise rather than one single scalar shared by every parameter.

5 Hyperparameters and tuning

The optimizer has several values that affect its behaviour. These values are called hyperparameters because they are selected before or during experiments rather than learned directly as model parameters.

Parameter	Role	Plain explanation
α / lr	Learning rate	Sets the base update size.
β_1	First moment decay	Controls how quickly the gradient average changes.
β_2	Second moment decay	Controls how quickly the squared-gradient average changes.
β_3	Third moment decay	Controls the memory of the skewness-related estimate.
β_4	Fourth moment decay	Controls the memory of the kurtosis-related estimate.
a	Skewness coefficient	Sets the influence of skewness on R_t.
b	Kurtosis coefficient	Sets the influence of excess kurtosis on R_t.
η	Gamma smoothing	Controls how much the previous gamma is retained.
$\gamma_{\min}, \gamma_{\max}$	Clipping limits	Restrict the raw modulation value.
ϵ	Numerical stability	Avoids division by zero.

Table 1: AdaGamma hyperparameters and their roles.

5.1 Why tuning was needed

AdaGamma introduces more choices than Adam. In particular, a, b, β_3, β_4 and η affect the new higher-order part of the algorithm. A value that works well on one test may not be best on another.

The project files contain two random-search approaches:

- `Tests_for_hyperparameter_WIR.py` samples values for β_3, β_4, a, b and η, then evaluates each candidate on a well-conditioned quadratic, an ill-conditioned quadratic and Rosenbrock.
- `hparam_search.py` samples learning rate, β_3, β_4, a, b and gamma smoothing, then evaluates each candidate using the MNIST training function.

5.2 How the synthetic-problem search works

The WIR search script has a maximum time budget of four hours and a maximum of 700 trials. It uses fixed problem instances and fixed random seeds to make comparisons repeatable. Each candidate is run on all three test functions.

The script counts the number of iterations needed to reach a gradient-norm tolerance of 10^{-6}. If a run does not converge before its iteration limit, it receives a penalty. The three step counts are then combined into a score. Lower is better.

The script also has target step counts:

- well-conditioned quadratic: 300 steps;
- ill-conditioned quadratic: 5000 steps;
- Rosenbrock: 5000 steps.

The search stops early if all three targets are met, or when the time/trial budget is reached.

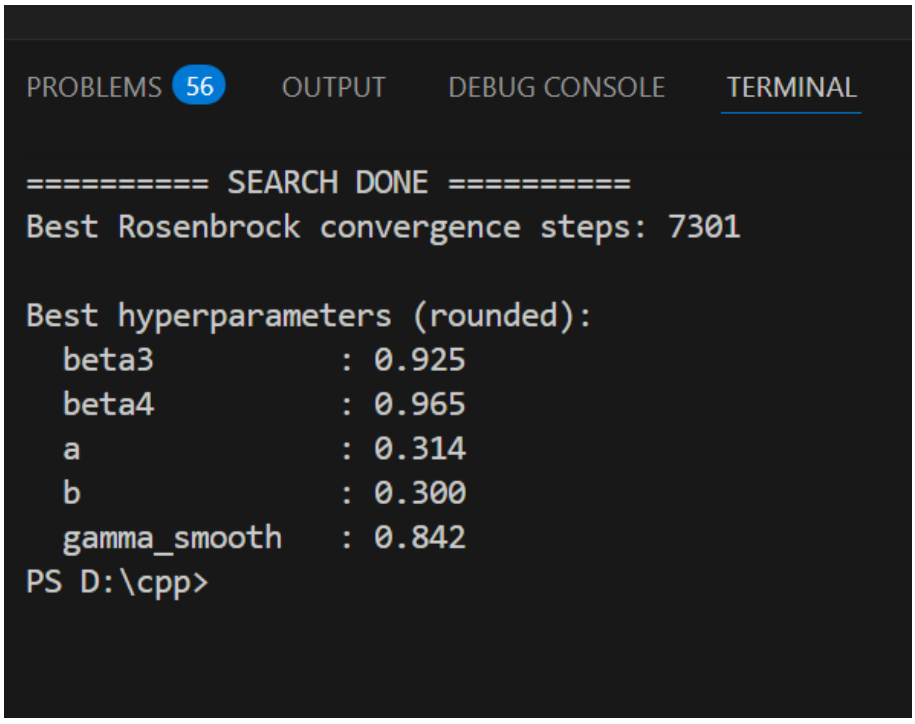


Figure 1: A saved terminal screenshot showing one best-parameter result from a search run.

The screenshot records the following rounded values: $\beta_3 = 0.925$, $\beta_4 = 0.965$, $a = 0.314$, $b = 0.300$ and $\eta = 0.842$. The screenshot reports 7301 steps for the best Rosenbrock result in that run. This is a search-run result, not the same as the final report's 10-run mean.

6 Testing the optimizer

We tested AdaGamma against three optimizers:

- SGD with momentum;
- Adam;
- RMSprop.

The final report says the benchmark experiments used 10 random seeds and reported mean and standard deviation. The tests were chosen to cover different types of optimization problems.

6.1 Well-conditioned quadratic

A quadratic function has the form:

$$f(x) = \frac{1}{2}x^T Qx, \qquad \nabla f(x) = Qx.$$

The matrix Q is symmetric and positive definite. In the test code, it is built using an orthogonal matrix and a set of positive eigenvalues.

A well-conditioned problem has eigenvalues that are not spread very far apart. This makes it comparatively easier for an optimizer to move towards the minimum.

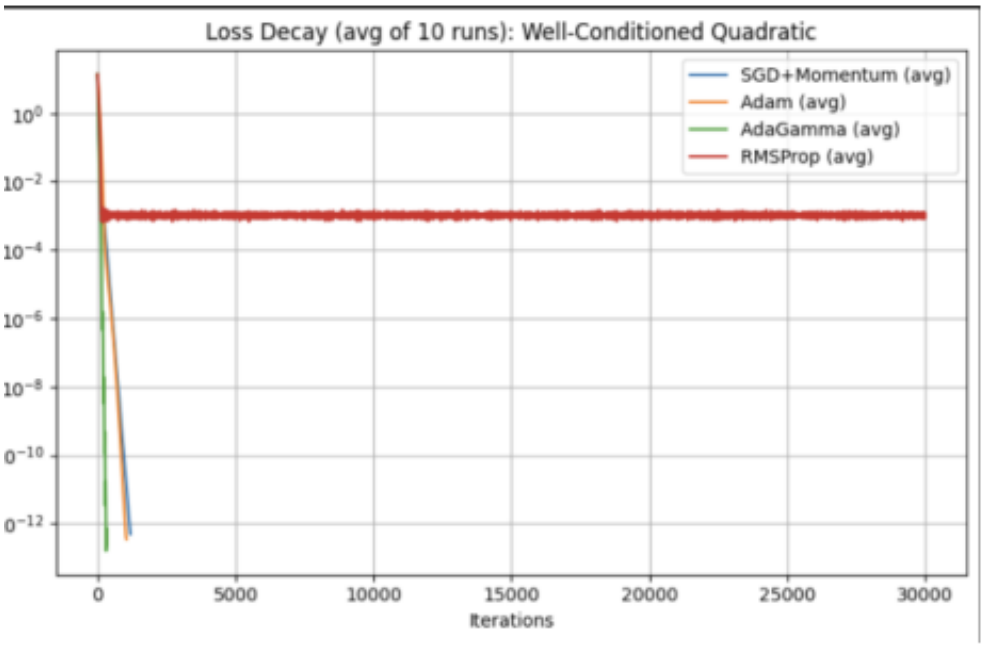


Figure 2: Loss against iteration for the well-conditioned quadratic test. Source: final project report, page 5.

Optimizer	Iterations to reach $f(x) < 10^{-6}$
AdaGamma	327 ± 5
Adam	1002 ± 38
SGD + Momentum	1186 ± 1
RMSprop	Did not converge within 30000

Table 2: Well-conditioned quadratic results reported in the project report.

In this test, AdaGamma reached the target in fewer iterations than the other methods. The report states that this was about 67% faster than Adam and 72.4% faster than SGD with momentum.

6.2 Ill-conditioned quadratic

An ill-conditioned quadratic has eigenvalues spread over a larger range. This creates directions with very different curvature. An optimizer may make progress quickly in one direction but slowly in another.

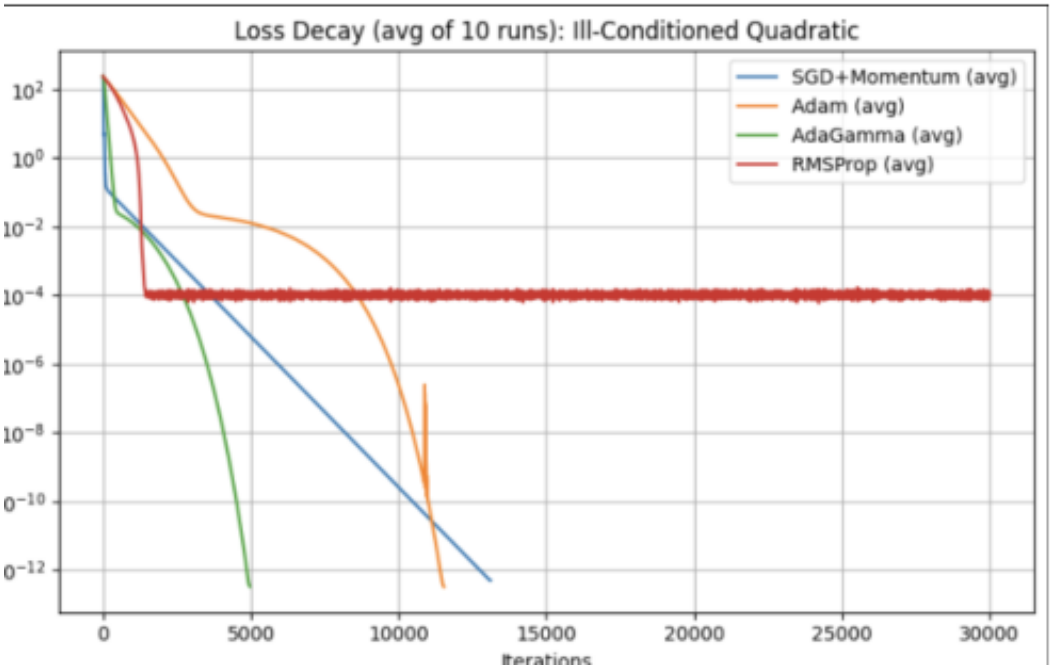


Figure 3: Loss against iteration for the ill-conditioned quadratic test. Source: final project report, page 6.

Optimizer	Iterations to reach $f(x) < 10^{-6}$
AdaGamma	4939 ± 14
Adam	11491 ± 27
SGD + Momentum	13087 ± 14
RMSprop	Did not converge within 30000

Table 3: Ill-conditioned quadratic results reported in the project report.

AdaGamma again reached the target in fewer iterations. The report gives improvements of about 57% over Adam and 62% over SGD with momentum.

6.3 Rosenbrock function

The Rosenbrock function is:

$$f(x, y) = (1 - x)^2 + 100(y - x^2)^2.$$

Its minimum is at $(1, 1)$, where the function value is zero. The important feature is its narrow curved valley. An optimizer has to move along this valley to reach the minimum. This makes it a useful test for checking how an optimizer behaves on a more difficult loss surface.

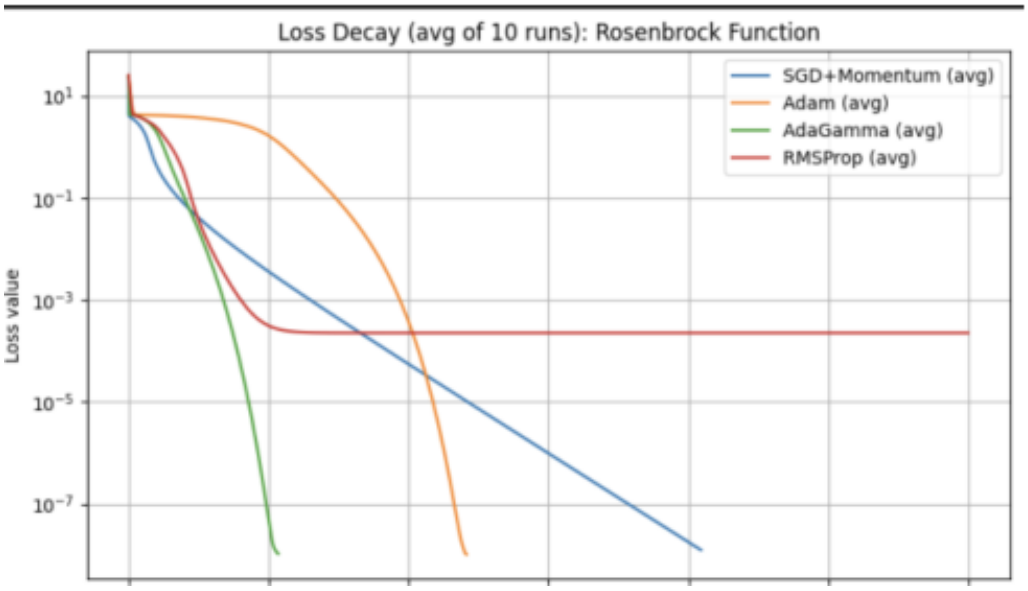


Figure 4: Loss against iteration for the Rosenbrock test. Source: final project report, page 7.

Optimizer	**Iterations to reach** $f(x) < 10^{-6}$
AdaGamma	5095 ± 111
Adam	11808 ± 174
SGD + Momentum	20435 ± 7
RMSprop	Did not converge within 30000

Table 4: Rosenbrock results reported in the project report.

The report states that AdaGamma was about 57% faster than Adam for this test.

6.4 MNIST digit classification

The final report also tests the optimizer on MNIST, a dataset of handwritten digits from 0 to 9. This is different from the previous tests because it involves training a neural network rather than minimizing a small mathematical function.

The report describes a feedforward neural network with:

$$784 \longrightarrow 128 \longrightarrow 10.$$

The 784 inputs correspond to the pixels of a 28×28 image. The hidden layer has 128 units and the output has 10 units, one for each digit class. The report uses 56,000 training samples, 14,000 test samples, a batch size of 64 and 25 epochs.

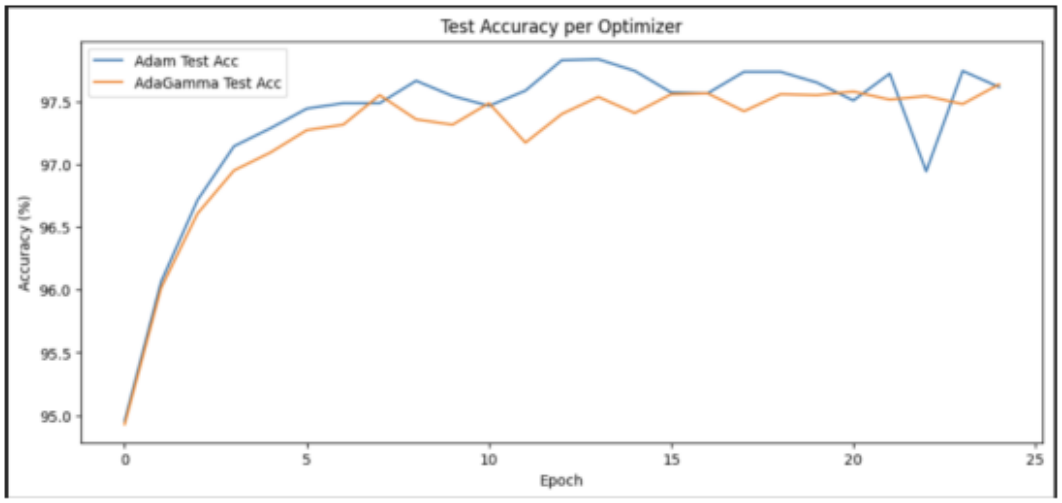


Figure 5: Test accuracy over 25 epochs for Adam and AdaGamma. Source: final project report, page 8.

Metric	AdaGamma	Adam
Test accuracy	$97.23\% \pm 0.59\%$	$97.36\% \pm 0.62\%$
Training accuracy	$99.26\% \pm 1.06\%$	$99.24\% \pm 1.07\%$
Seconds per epoch	13.320 ± 2.185	3.431 ± 0.159

Table 5: MNIST results reported in the final project report.

The main result is that AdaGamma and Adam reached almost the same accuracy. However, AdaGamma took considerably longer per epoch in this experiment. So the improvement seen on the mathematical test functions did not carry over to better MNIST accuracy or training speed.

7 What the results tell us

Across the three mathematical test functions, the final report shows that AdaGamma needed fewer iterations to reach the chosen loss threshold than Adam and SGD with momentum. RMSprop did not reach the threshold within the 30,000-iteration limit in these reported tests.

MNIST gave a different result. AdaGamma matched Adam closely in accuracy, but was slower. This is important: an optimizer that works well on controlled mathematical functions is not automatically better for neural-network training.

The extra moment calculations also have a cost. AdaGamma stores and updates two additional moment tensors, and performs extra element-wise operations to calculate skewness, kurtosis and gamma.

8 Limitations and possible improvements

The final report identifies three main limitations.

8.1 Performance on neural-network training

AdaGamma performs well on the quadratic and Rosenbrock tests, but does not improve MNIST accuracy over Adam. The report suggests combining the higher-order modulation with adaptive β_1 and β_2, and using mini-batch normalisation of the higher moments.

8.2 Extra computation and memory

The third and fourth moments require extra state and operations. This may become important for large models or GPUs with limited memory. One possible direction is to approximate skewness and kurtosis using lower-cost estimates.

8.3 More hyperparameters

AdaGamma has extra parameters such as a, b, η, β_3 and β_4. The best values can depend on the problem. Automatic tuning or better general-purpose defaults would make the optimizer easier to use.

9 My contribution and what I learned

The final report specifically records that I worked closely on the mathematical aspects of the algorithm. All four members were involved in ideation and testing. Based on the project files and this contribution record, my work can be described in the following way:

- Worked on the mathematical design of AdaGamma, including the use of higher-order gradient moments.
- Developed an understanding of how the first, second, third and fourth moments fit into the update rule.
- Helped translate the mathematical update into an optimizer implementation.
- Worked on testing and tuning hyperparameters across the benchmark problems.
- Compared the results with SGD with momentum, Adam and RMSprop, and interpreted where the optimizer performed well and where it did not.

The code in the archive was generated with substantial AI assistance, so this log does not claim that every implementation detail was typed or programmed manually. My focus was on understanding the method, developing the mathematical idea, checking how it was represented in code, and interpreting the experiments.

10 Notes about the submitted files

The project archive contains several notebooks and scripts from different stages of development. There are a few differences worth recording so that the results are not mixed up.

1. **Different AdaGamma defaults:** the Python class defaults to $\beta_3 = 0.95$, $\beta_4 = 0.99$, $a = 0.2$, $b = 0.05$ and $\eta = 0.9$. The final report lists another set of defaults, and the benchmark notebooks use values such as $\beta_3 = 0.93$, $\beta_4 = 0.97$, $a = 0.3$, $b = 0.3$ and $\eta = 0.84$. This is consistent with values being changed during tuning, but the archive does not identify one universal setting for every experiment.

2. **Two MNIST implementations:** the archive contains a PyTorch script that uses logistic regression (784 inputs directly to 10 outputs) for 10 epochs. The final report's MNIST experiment instead describes a network with a 128-unit hidden layer and 25 epochs. The results in this log use the final report's stated experiment and numbers.
3. **Different experiment outputs:** some notebooks contain individual runs or later test runs whose iteration counts differ from the report. The tables in this log use the final report's mean and standard deviation values. The notebook outputs are useful as development records, but they should not be mixed with the report's final summary.
4. **Search screenshot:** the saved screenshot shows one hyperparameter-search outcome. It is not the same as the final benchmark table.

11 Conclusion

AdaGamma is an extension of Adam that uses skewness and excess kurtosis to modify the size of each update. The main mathematical contribution is the addition of third- and fourth-moment tracking and the smoothed modulation factor γ.

In the reported tests, AdaGamma converged faster than Adam on the well-conditioned quadratic, ill-conditioned quadratic and Rosenbrock problems. On MNIST, it reached nearly the same accuracy as Adam but took longer to train. The project therefore showed that higher-order gradient information can be useful on some optimization landscapes, but it also introduced extra computation and more hyperparameters to tune.

The main thing I take away from this project is how a mathematical idea can be added to an existing optimizer, implemented as a reusable class, and tested against standard methods. It also showed why an optimizer has to be tested on more than one type of problem before claiming that it is generally better.

References

1. D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," ICLR, 2015.
2. S. J. Reddi, S. Kale and S. Kumar, "On the convergence of Adam and beyond," ICLR, 2018.
3. I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," ICLR, 2019.
4. M. Zaheer et al., "Adaptive methods for nonconvex optimization," NeurIPS, 2018.
5. J. Zhuang et al., "AdaBelief optimizer: Adapting stepsizes by the belief in observed gradients," NeurIPS, 2020.
6. Group 34, "AdaGamma Optimizer: An Adam Extension Using Higher-Order Gradient Moments," project report, FM216, Monsoon 2025.
7. Group 34, "AdaGamma: An Adam Extension Using Higher-Order Gradient Moments," project presentation/poster, Optimization Course, Fall 2025.

Index

AdaGamma, 4
Adam, 4

benchmark, 11

gradient, 5

hyperparameter, 9

kurtosis, 5

limitations, 14

MNIST, 13
moment, 5

optimizer, 4

Python, 7

quadratic function, 11

Rosenbrock function, 13

skewness, 5